

Aula 02 - Dados crescendo no procedural

Contexto de negócio

Regra de compliance: conta inativa e limite operacional precisam bloquear operações automaticamente.

Quando a conta ganha novos dados (limite, status), como manter regras sem se perder em parâmetros?

Tensão operacional

Com mais campos por conta, métodos passam a receber muitos parâmetros.

Objetivo técnico da entrega

- Regra com vários dados de entrada.
- Validação combinada (`saldo`, `limite`, `ativa`, `valor`).

Recurso técnico desta aula

Recurso inserido: `podeSacar` e `sacarComLimite` com parâmetros explícitos para separar decisão da operação.

Valor prático entregue

- Evita operações irregulares em contas bloqueadas.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execucao em Java.
public class AppGestaoContas {
    static boolean podeSacar(double p_saldo, double p_limite, boolean p_ativa,
double p_valor) {
        return p_ativa && p_valor > 0 && p_valor <= (p_saldo + p_limite);
    }

    static double sacarComLimite(double p_saldo, double p_limite, boolean p_ativa,
double p_valor) {
        if (podeSacar(p_saldo, p_limite, p_ativa, p_valor)) {
            return p_saldo - p_valor;
        }
        return p_saldo;
    }
}
```

```
}

    static void exibirConta(String p_titular, double p_saldo, double p_limite,
boolean p_ativa) {
        System.out.println(
            "Titular: " + p_titular
            + " | Saldo: " + p_saldo
            + " | Limite: " + p_limite
            + " | Ativa: " + p_ativa
        );
    }

    public static void main(String[] args) {
        // Aula 02: dados crescem e os parâmetros da regra aumentam.
        String titularAna = "Ana";
        double saldoAna = 1000.00;
        double limiteAna = 500.00;
        boolean ativaAna = true;

        String titularBruno = "Bruno";
        double saldoBruno = 300.00;
        double limiteBruno = 100.00;
        boolean ativaBruno = true;

        saldoAna = sacarComLimite(saldoAna, limiteAna, ativaAna, 1200.00);
        saldoBruno = sacarComLimite(saldoBruno, limiteBruno, ativaBruno, 450.00);

        exibirConta(titularAna, saldoAna, limiteAna, ativaAna);
        exibirConta(titularBruno, saldoBruno, limiteBruno, ativaBruno);
    }
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Discussão técnica: condição booleana composta

Trecho-chave desta aula:

```
return p_ativa && p_valor > 0 && p_valor <= (p_saldo + p_limite);
```

Esse retorno condensa três regras de negócio em uma única expressão.

Quando isso é bom:

- deixa a regra objetiva e centralizada;
- reduz ruído de `if` aninhado;
- facilita leitura quando as condições são claras.

Quando pode piorar:

- se a expressão crescer demais, a legibilidade cai;
- fica difícil explicar qual parte falhou;
- manutenção e depuração ficam mais caras.

Um bom programador (e um bom algoritmo) escolhe o nível certo de complexidade:

- compacto o suficiente para ser elegante;
- explícito o suficiente para ser auditável.

Se a regra começar a crescer, uma estratégia melhor é quebrar em variáveis intermediárias com nomes de domínio.

Discussão de contrato: o que significa `return p_saldo`

No método abaixo, existem dois resultados possíveis:

```
static double sacarComLimite(double p_saldo, double p_limite, boolean p_ativa,
double p_valor) {
    if (podeSacar(p_saldo, p_limite, p_ativa, p_valor)) {
        return p_saldo - p_valor;
    }
    return p_saldo;
}
```

Esse contrato diz:

- se o saque for permitido, retorna o novo saldo;
- se o saque for negado, retorna o saldo original.

Faz sentido? Sim, desde que a equipe entenda que o método sempre devolve um saldo válido e que a negativa não gera exceção nem mensagem automática.

Ponto de atenção importante:

- o chamador precisa saber se houve saque ou não;
- apenas olhar o valor retornado pode esconder a falha quando o saldo não muda.

Exemplo de risco lógico:

- saldo inicial: `300`;
- tentativa de saque: `450`;
- retorno: `300`;
- se o fluxo não comparar antes/depois nem registrar a negativa, o erro operacional fica silencioso.

Como reduzir esse risco no uso:

- comparar `saldoAntes` e `saldoDepois` quando a operação for crítica;
- usar `podeSacar(...)` para decisão explícita antes da atualização;
- padronizar mensagem de sucesso/negativa no ponto de chamada.

Alternativas de design para discussão em equipe:

- manter este contrato simples (boa didática para início de curso);
- retornar `boolean` e atualizar saldo por outro mecanismo;
- criar um resultado mais explícito em etapas futuras (ex.: `SaqueResultado` com `aprovado`, `saldoFinal`, `motivo`).

Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não “conhece” a conta por contexto implícito.
- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;
- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Na POO, estado e comportamento passam a coexistir no objeto.

Perguntas de decisão

1. Quantos dados a regra de saque precisa agora?
2. Qual risco de passar parâmetros fora de ordem?
3. Quando a expressão booleana composta fica clara e quando fica complexa demais?
4. O que o `return p_saldo` comunica sobre saque negado?
5. Em quais cenários isso pode ocultar erro de lógica no chamador?
6. Quem junta os dados da conta no procedural?

Variações de cenário

1. Tornar `ativaBruno = false` e testar saque.
2. Trocar `limiteBruno` para 300 e testar novamente.

Exercícios progressivos

1. Criar regra `podeDepositar(boolean ativa, double valor)`.
2. Impedir operações quando conta estiver inativa.
3. No `main`, comparar `saldoAntes` e `saldoDepois` para detectar saque negado sem depender de mensagem pronta.

Desafio de equipe

Adicionar campo `taxaSaque` por conta e considerar taxa no cálculo de `podeSacar`.

Debater prós e contras de três contratos para saque:

1. Retornar saldo final (modelo atual).
2. Retornar boolean de aprovação.
3. Retornar objeto de resultado com motivo da negativa.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.
- A condição composta permanece compreensível ou foi decomposta de forma clara.
- O fluxo consegue distinguir saque aprovado de saque negado sem ambiguidade.

Fechamento executivo

Os dados cresceram e a assinatura dos métodos ficou mais pesada. Na próxima, vamos organizar várias contas com arrays paralelos.

Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?
2. Risco real: qual erro operacional essa implementação reduz?
3. Risco de contrato: quando `return p_saldo` ajuda e quando pode esconder problema?
4. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de `AppGestaoContas.java` no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_booleans.asp
- https://www.w3schools.com/java/java_if_else.asp

- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>